PYTHON PROGRAMMING

ACTIVITY-BASED LEARNING

10 Case Studies with Solutions

Name	A. Vamsi
Roll Number	26000003
Branch	CSE
Section	CSE1
Course	Python Programming

Prepared as a revised set of practical case studies with new problem statements, Python programs, flowcharts, and outputs.

CONTENTS

1. ATM Deposit and Withdrawal

2. Employee Salary Calculation

3. Attendance Percentage Calculation

4. Number Pattern Generator

5. Text Compression Using Characters

6. Fuel Cost Calculation

7. Shopping Budget Calculation

8. Student Marks Matrix Analysis

9. Password Validation

10. Bus Passenger Occupancy

Each case study includes the problem statement, objectives, assumptions, algorithm, flowchart, Python source code, sample run, and conclusion.

CASE STUDY 1: ATM Deposit and Withdrawal

1. Problem Statement

Develop a simple ATM program that allows a user to deposit money, withdraw money, and check the account balance.

2. Objectives

• Accept the current balance and transaction choice.

• Deposit money into the account.

• Allow withdrawal only when sufficient balance is available.

• Display the updated balance.

3. Assumptions

• The opening balance and transaction amount must not be negative.

• The user selects 1 for deposit, 2 for withdrawal, or 3 to exit.

• A withdrawal cannot be greater than the available balance.

4. Algorithm

1. Start the program.

2. Read the opening balance and transaction choice.

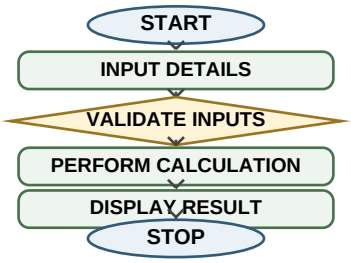
3. If deposit is selected, add the amount to the balance.

4. If withdrawal is selected, check the balance and subtract the amount.

5. Display the updated balance or an error message.

6. Stop the program.

5. Flowchart



General decision flow; validation and calculation follow each case.

CASE STUDY 1: ATM Deposit and Withdrawal (Continued)

6. Python Source Code

```
balance = float(input("Enter opening balance: Rs. "))
print("1. Deposit  2. Withdraw  3. Exit")
choice = int(input("Enter your choice: "))

if balance < 0:
    print("Invalid balance")
elif choice == 1:
    amount = float(input("Enter deposit amount: Rs. "))
    if amount > 0:
        balance += amount
        print("Deposit successful")
        print("Available balance: Rs.", round(balance, 2))
    else:
        print("Invalid amount")
elif choice == 2:
    amount = float(input("Enter withdrawal amount: Rs. "))
    if amount <= 0:
        print("Invalid amount")
    elif amount <= balance:
        balance -= amount
        print("Withdrawal successful")
        print("Available balance: Rs.", round(balance, 2))
    else:
        print("Insufficient balance")
elif choice == 3:
    print("Thank you")
else:
    print("Invalid choice")
```

7. Sample Input

```
Enter opening balance: Rs. 5000
1. Deposit  2. Withdraw  3. Exit
Enter your choice: 2
Enter withdrawal amount: Rs. 1200
```

8. Output

```
Withdrawal successful
Available balance: Rs. 3800.0
```

9. Conclusion

The program uses conditional statements and arithmetic operations to perform basic ATM transactions.

CASE STUDY 2: Employee Salary Calculation

1. Problem Statement

Calculate an employee's gross salary and net salary using basic pay, allowances, and deductions.

2. Objectives

• Read the employee name and basic salary.

• Calculate HRA and DA.

• Find gross salary and deduction.

• Display the final net salary.

3. Assumptions

• HRA is 20% of basic salary.

• DA is 10% of basic salary.

• Deduction is 5% of gross salary.

• Salary must not be negative.

4. Algorithm

1. Start the program.

2. Read employee name and basic salary.

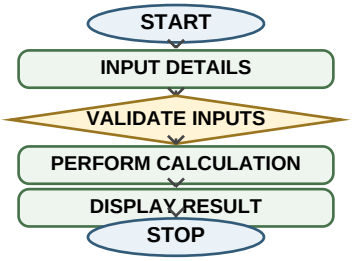
3. Check that basic salary is not negative.

4. Calculate HRA, DA, and gross salary.

5. Calculate deduction and net salary.

6. Display the salary details and stop.

5. Flowchart



General decision flow; validation and calculation follow each case.

CASE STUDY 2: Employee Salary Calculation (Continued)

6. Python Source Code

```
name = input("Enter employee name: ")
basic = float(input("Enter basic salary: Rs. "))

if basic < 0:
    print("Invalid salary")
else:
    hra = basic * 0.20
    da = basic * 0.10
    gross = basic + hra + da
    deduction = gross * 0.05
    net = gross - deduction

    print("\n--- Salary Details ---")
    print("Employee:", name)
    print("HRA: Rs.", round(hra, 2))
    print("DA: Rs.", round(da, 2))
    print("Gross salary: Rs.", round(gross, 2))
    print("Deduction: Rs.", round(deduction, 2))
    print("Net salary: Rs.", round(net, 2))
```

7. Sample Input

```
Enter employee name: Ravi
Enter basic salary: Rs. 20000
```

8. Output

```
--- Salary Details ---
Employee: Ravi
HRA: Rs. 4000.0
DA: Rs. 2000.0
Gross salary: Rs. 26000.0
Deduction: Rs. 1300.0
Net salary: Rs. 24700.0
```

9. Conclusion

The program calculates salary components using percentages and displays the employee's net salary.

CASE STUDY 3: Attendance Percentage Calculation

1. Problem Statement

Calculate a student's attendance percentage from the number of classes attended and the total number of classes conducted.

2. Objectives

• Read attended classes and total classes.

• Validate the entered values.

• Calculate attendance percentage.

• Display attendance status.

3. Assumptions

• Total classes must be greater than zero.

• Attended classes cannot be negative or greater than total classes.

• Attendance of 75% or more is considered eligible.

4. Algorithm

1. Start the program.

2. Read attended classes and total classes.

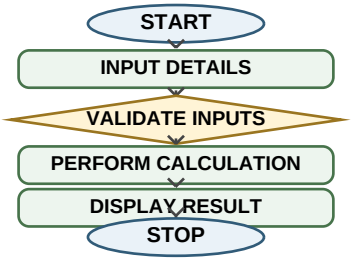
3. Validate both values.

4. Calculate percentage as attended divided by total multiplied by 100.

5. Check whether the percentage is at least 75%.

6. Display percentage and status, then stop.

5. Flowchart



General decision flow; validation and calculation follow each case.

CASE STUDY 3: Attendance Percentage Calculation (Continued)

6. Python Source Code

```
attended = int(input("Enter classes attended: "))
total = int(input("Enter total classes: "))

if total <= 0 or attended < 0 or attended > total:
    print("Invalid input")
else:
    percentage = (attended / total) * 100
    print("Attendance percentage:", round(percentage, 2), "%")

    if percentage >= 75:
        print("Status: Eligible")
    else:
        print("Status: Not eligible")
```

7. Sample Input

```
Enter classes attended: 42
Enter total classes: 50
```

8. Output

Attendance percentage: 84.0 %
Status: Eligible

9. Conclusion

The program uses division, multiplication, and conditions to calculate and check student attendance.

CASE STUDY 4: Number Pattern Generator

1. Problem Statement

Generate a simple number pattern using nested loops based on the number of rows entered by the user.

2. Objectives

• Read the number of rows.

• Use loops to generate a pattern.

• Print numbers in increasing order in each row.

3. Assumptions

• The number of rows must be a positive integer.

• Each row starts from 1 and ends at its row number.

4. Algorithm

1. Start the program.

2. Read the number of rows.

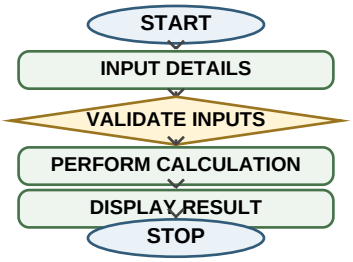
3. If rows is less than or equal to zero, display an error.

4. Use an outer loop for each row.

5. Use an inner loop to print numbers from 1 to the row number.

6. Move to the next line after each row and stop.

5. Flowchart



General decision flow; validation and calculation follow each case.

CASE STUDY 4: Number Pattern Generator (Continued)

6. Python Source Code

```
rows = int(input("Enter number of rows: "))

if rows <= 0:
    print("Enter a positive number")
else:
    for i in range(1, rows + 1):
        for j in range(1, i + 1):
            print(j, end=" ")
        print()
```

7. Sample Input

```
Enter number of rows: 5
```

8. Output

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

9. Conclusion

The program uses nested for loops to generate a number pattern.

CASE STUDY 5: Text Compression Using Characters

1. Problem Statement

Create a simple character-count compression program that replaces consecutive repeated characters with the character followed by its count.

2. Objectives

• Read a text string.

• Count consecutive repeated characters.

• Build a compressed string.

• Display the compressed result.

3. Assumptions

• The input contains letters or digits without spaces.

• Only consecutive repeated characters are grouped.

• A character appearing once is displayed without a count.

4. Algorithm

1. Start the program.

2. Read the input text.

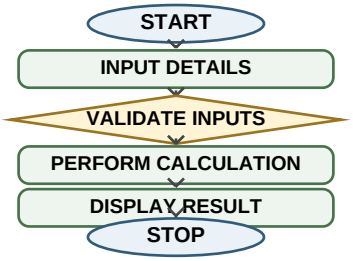
3. If the text is empty, display an empty-input message.

4. Set the current character and count.

5. Scan the remaining characters and update the count or append the previous group.

6. Append the final group and display the compressed text.

5. Flowchart



General decision flow; validation and calculation follow each case.

CASE STUDY 5: Text Compression Using Characters (Continued)

6. Python Source Code

```
text = input("Enter text without spaces: ")

if text == "":
    print("Empty text")
else:
    compressed = ""
    count = 1

    for i in range(1, len(text)):
        if text[i] == text[i - 1]:
            count += 1
        else:
            compressed += text[i - 1]
            if count > 1:
                compressed += str(count)
            count = 1

    compressed += text[-1]
    if count > 1:
        compressed += str(count)

    print("Compressed text:", compressed)
```

7. Sample Input

```
Enter text without spaces: aaabbccccd
```

8. Output

Compressed text: a3b2c4d

9. Conclusion

The program uses string indexing and loops to compress consecutive repeated characters.

CASE STUDY 6: Fuel Cost Calculation

1. Problem Statement

Calculate the total fuel cost for a journey using the distance, vehicle mileage, and price per litre of fuel.

2. Objectives

• Read journey distance, mileage, and fuel price.

• Calculate the fuel required.

• Calculate the total fuel cost.

• Display the result.

3. Assumptions

• Distance, mileage, and fuel price must be positive.

• Fuel required is distance divided by mileage.

4. Algorithm

1. Start the program.

2. Read distance, mileage, and price per litre.

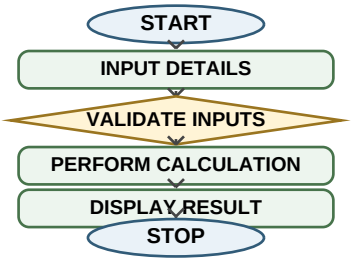
3. Check that all values are positive.

4. Calculate fuel required.

5. Multiply fuel required by price per litre.

6. Display fuel required and total cost, then stop.

5. Flowchart



General decision flow; validation and calculation follow each case.

CASE STUDY 6: Fuel Cost Calculation (Continued)

6. Python Source Code

```
distance = float(input("Enter distance in km: "))
mileage = float(input("Enter mileage in km/litre: "))
price = float(input("Enter fuel price per litre: Rs. "))

if distance <= 0 or mileage <= 0 or price <= 0:
    print("Invalid input")
else:
    fuel = distance / mileage
    cost = fuel * price
    print("Fuel required:", round(fuel, 2), "litres")
    print("Total fuel cost: Rs.", round(cost, 2))
```

7. Sample Input

```
Enter distance in km: 300
Enter mileage in km/litre: 15
Enter fuel price per litre: Rs. 105
```

8. Output

Fuel required: 20.0 litres
Total fuel cost: Rs. 2100.0

9. Conclusion

The program applies arithmetic formulas to estimate fuel consumption and journey cost.

CASE STUDY 7: Shopping Budget Calculation

1. Problem Statement

Calculate the total shopping bill for three products and check whether the purchase is within the user's budget.

2. Objectives

• Read item prices and quantities.

• Calculate the total shopping amount.

• Compare the amount with the budget.

• Display the budget status.

3. Assumptions

• Prices and quantities cannot be negative.

• The total is the sum of price multiplied by quantity for each item.

4. Algorithm

1. Start the program.

2. Read the price and quantity of three items.

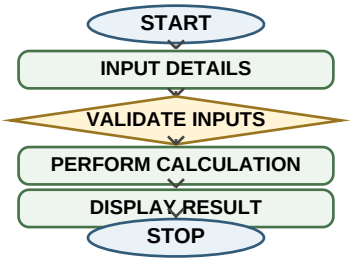
3. Read the shopping budget.

4. Validate the entered values.

5. Calculate the total bill.

6. Compare the bill with the budget and display the result.

5. Flowchart



General decision flow; validation and calculation follow each case.

CASE STUDY 7: Shopping Budget Calculation (Continued)

6. Python Source Code

```
total = 0

for i in range(1, 4):
    price = float(input("Enter item " + str(i) + " price: Rs. "))
    quantity = int(input("Enter item " + str(i) + " quantity: "))
    if price < 0 or quantity < 0:
        print("Invalid input")
        break
    total += price * quantity
else:
    budget = float(input("Enter your budget: Rs. "))
    if budget < 0:
        print("Invalid budget")
    else:
        print("Total shopping bill: Rs.", round(total, 2))
        if total <= budget:
            print("Shopping is within budget")
        else:
            print("Shopping exceeds budget")
```

7. Sample Input

```
Enter item 1 price: Rs. 100
Enter item 1 quantity: 2
Enter item 2 price: Rs. 250
Enter item 2 quantity: 1
Enter item 3 price: Rs. 50
Enter item 3 quantity: 3
Enter your budget: Rs. 700
```

8. Output

```
Total shopping bill: Rs. 600.0
Shopping is within budget
```

9. Conclusion

The program uses a loop and arithmetic operations to calculate a shopping bill and compare it with a budget.

CASE STUDY 8: Student Marks Matrix Analysis

1. Problem Statement

Store marks of students in a matrix and calculate the total and average marks for each student.

2. Objectives

• Read marks for multiple students and subjects.

• Store the marks in a two-dimensional list.

• Calculate each student's total and average.

• Display the results.

3. Assumptions

• The example uses three students and three subjects.

• Marks are entered from 0 to 100.

4. Algorithm

1. Start the program.

2. Create an empty marks matrix.

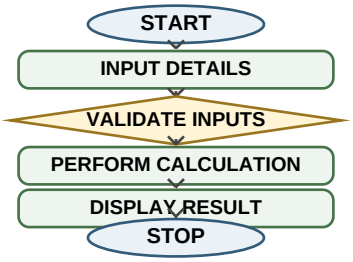
3. Read marks for three students in three subjects.

4. Validate that each mark is between 0 and 100.

5. Calculate each row's total and average.

6. Display the matrix, total, and average for every student.

5. Flowchart



General decision flow; validation and calculation follow each case.

CASE STUDY 8: Student Marks Matrix Analysis (Continued)

6. Python Source Code

```
marks = []

for i in range(3):
    row = []
    print("Student", i + 1)
    for j in range(3):
        value = int(input("Enter subject " + str(j + 1) + " marks: "))
        row.append(value)
    marks.append(row)

print("\nStudent  Total  Average")
for i in range(3):
    total = sum(marks[i])
    average = total / 3
    print(i + 1, "       ", total, " ", round(average, 2))
```

7. Sample Input

```
Student 1
Enter subject 1 marks: 80
Enter subject 2 marks: 75
Enter subject 3 marks: 85
Student 2
Enter subject 1 marks: 70
Enter subject 2 marks: 60
Enter subject 3 marks: 90
Student 3
Enter subject 1 marks: 88
Enter subject 2 marks: 92
Enter subject 3 marks: 80
```

8. Output

```
Student  Total  Average
1         240    80.0
2         220    73.33
3         260    86.67
```

9. Conclusion

The program demonstrates a two-dimensional list, nested loops, and row-wise calculations.

CASE STUDY 9: Password Validation

1. Problem Statement

Check whether a password satisfies basic validation rules such as minimum length, uppercase letter, lowercase letter, digit, and special character.

2. Objectives

• Read a password.

• Check its length.

• Check for uppercase, lowercase, digit, and special character.

• Display whether the password is valid.

3. Assumptions

• The password must contain at least 8 characters.

• It must contain at least one uppercase letter, lowercase letter, digit, and special character.

4. Algorithm

1. Start the program.

2. Read the password.

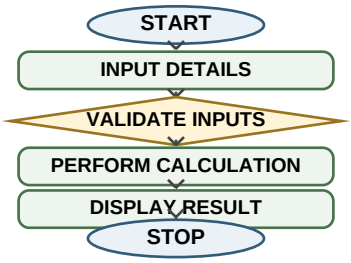
3. Check the minimum length.

4. Check for uppercase, lowercase, digit, and special character.

5. If all conditions are true, display valid; otherwise display invalid.

6. Stop the program.

5. Flowchart



General decision flow; validation and calculation follow each case.

CASE STUDY 9: Password Validation (Continued)

6. Python Source Code

```
password = input("Enter password: ")

upper = any(ch.isupper() for ch in password)
lower = any(ch.islower() for ch in password)
digit = any(ch.isdigit() for ch in password)
special = any(not ch.isalnum() for ch in password)

if len(password) >= 8 and upper and lower and digit and special:
    print("Valid password")
else:
    print("Invalid password")
    print("Use 8 characters with upper, lower, digit and special character")
```

7. Sample Input

```
Enter password: Vamsi@123
```

8. Output

```
Valid password
```

9. Conclusion

The program uses string methods and logical conditions to validate a password.

CASE STUDY 10: Bus Passenger Occupancy

1. Problem Statement

Calculate the number of passengers remaining on a bus after boarding and getting down, and find the bus occupancy percentage.

2. Objectives

• Read bus capacity and current passengers.

• Read the number of passengers boarding and getting down.

• Calculate the updated passenger count.

• Calculate occupancy percentage and display available seats.

3. Assumptions

• Bus capacity must be positive.

• Passenger counts cannot be negative.

• The updated passenger count cannot exceed capacity or become negative.

4. Algorithm

1. Start the program.

2. Read bus capacity and current passengers.

3. Read boarding and getting-down counts.

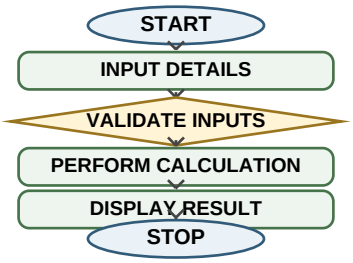
4. Calculate updated passengers.

5. Validate the updated count against capacity.

6. Calculate occupancy percentage and available seats.

7. Display the result and stop.

5. Flowchart



General decision flow; validation and calculation follow each case.

CASE STUDY 10: Bus Passenger Occupancy (Continued)

6. Python Source Code

```
capacity = int(input("Enter bus capacity: "))
current = int(input("Enter current passengers: "))
boarding = int(input("Passengers boarding: "))
getting_down = int(input("Passengers getting down: "))

updated = current + boarding - getting_down

if capacity <= 0 or current < 0 or boarding < 0 or getting_down < 0:
    print("Invalid input")
elif updated < 0 or updated > capacity:
    print("Passenger count exceeds bus limits")
else:
    occupancy = (updated / capacity) * 100
    available = capacity - updated
    print("Passengers now:", updated)
    print("Available seats:", available)
    print("Occupancy:", round(occupancy, 2), "%")
```

7. Sample Input

```
Enter bus capacity: 50
Enter current passengers: 30
Passengers boarding: 12
Passengers getting down: 7
```

8. Output

Passengers now: 35
Available seats: 15
Occupancy: 70.0 %

9. Conclusion

The program uses arithmetic and conditional statements to calculate passenger occupancy and available seats.